



Sprint 4: Modeling Avanzado Optimización, Ensamblés y Validación Final

Sprint Backlog, Roles y Definition of Done

Pedro Shiguihara, pshiguihara@pucp.edu.pe

24 de abril de 2026

Agenda

1. Contexto: del baseline al modelado avanzado
2. Metodología: CRISP-DM y Scrum aplicados al Sprint 4
3. Product Backlog: ítems del Sprint 4
4. Sprint 4: Alcance y Sprint Planning
5. Roles del equipo (5 personas)
6. Detalle de tareas por rol
7. Entregables del Sprint 4
8. Criterios de aceptación y Definition of Done

Contexto del Sprint 4

Del Sprint 3 (baseline) al Sprint 4 (optimizar)

🎯 Objetivo del Sprint 4

Optimizar los **hiperparámetros** de los modelos candidatos seleccionados en el Sprint 3, explorar **técnicas avanzadas** (ensambles, stacking) y **validar** el modelo final en el test set para confirmar su capacidad de generalización antes de la fase de Evaluation.

➡ Insumos del Sprint 3

- `09_baseline_models.ipynb` (baselines entrenados)
- `10_evaluation.ipynb` (métricas de cross-validation)
- `11_model_comparison.ipynb` (tabla comparativa y candidatos)
- `src/models.py` (funciones reutilizables)
- `models/baseline_*.pkl` (modelos persistidos)
- `models/experiments_log.csv` (registro de experimentos)
- Top 2-3 candidatos seleccionados y justificados

🔧 Herramientas Clave

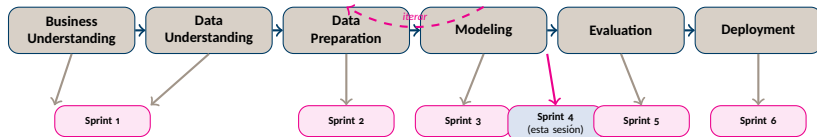
- Python 3.10+
- scikit-learn (GridSearchCV, RandomizedSearchCV)
- XGBoost / LightGBM (gradient boosting)
- Optuna (optimización bayesiana)
- joblib (persistir modelos)
- DVC (versionar modelos)
- Git + GitHub Projects

⚠ Clave

Este sprint **optimiza** los candidatos del Sprint 3 y produce el **modelo final** que será evaluado en términos de Business Value en el Sprint 5.

Metodología: CRISP-DM dentro de Scrum

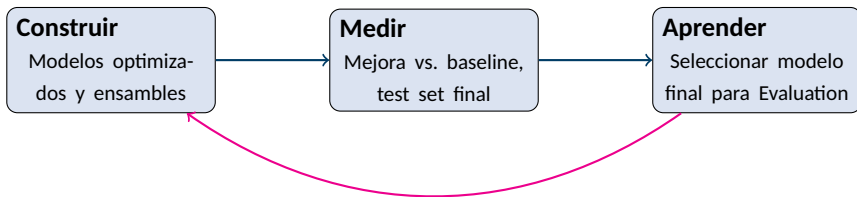
Mapeo de fases CRISP-DM a 6 Sprints — foco en Sprint 4



- **Sprint 1:** Business & Data Understanding.
- **Sprint 2:** Data Preparation.
- **Sprint 3:** Modeling baseline.
- **Sprint 4:** Modeling avanzado + tuning (*esta sesión*).
- **Sprint 5:** Evaluation + Business Value.
- **Sprint 6:** Deployment en AWS (MVP).

Ciclo Lean Startup aplicado al Sprint 4

Construir → Medir → Aprender



💡 Filosofía del Sprint 4

El objetivo es **exprimir el máximo rendimiento** de los candidatos seleccionados en el Sprint 3 mediante tuning de hiperparámetros y técnicas avanzadas. El modelo final de este sprint será el que se evalúe en términos de Business Value (Sprint 5) y se despliegue (Sprint 6).

Product Backlog Completo

Historias de usuario para todo el proyecto (6 sprints)

ID	Historia de Usuario	Fase CRISP-DM	Sprint
PB-01	Definir el problema de negocio y objetivos	Business Underst.	1
PB-02	Identificar criterios de éxito del modelo	Business Underst.	1
PB-03	Cargar y explorar el dataset inicial	Data Understanding	1
PB-04	Analizar calidad de datos (nulos, tipos, duplicados)	Data Understanding	1
PB-05	Generar estadísticas descriptivas y visualizaciones	Data Understanding	1
PB-06	Construir prototipo interactivo de exploración	Data Understanding	1
PB-07	Limpiar y transformar variables	Data Preparation	2
PB-08	Realizar feature engineering	Data Preparation	2
PB-09	Balancear clases si es necesario	Data Preparation	2
PB-10	Entrenar modelos baseline de clasificación	Modeling	3
PB-11	Evaluar modelos baseline con métricas	Modeling	3
PB-12	Comparar modelos y seleccionar candidatos	Modeling	3
PB-13	Optimizar hiperparámetros del mejor modelo	Modeling	4
PB-14	Aplicar técnicas avanzadas (ensambles, stacking)	Modeling	4
PB-15	Validar modelo final (cross-validation, test set)	Modeling	4
PB-16	Evaluar Business Value del modelo seleccionado	Evaluation	5
PB-17	Construir dashboard interactivo de resultados	Evaluation	5
PB-18	Documentar hallazgos y recomendaciones de negocio	Evaluation	5
PB-19	Empaquetar modelo como API (Flask/FastAPI)	Deployment	6
PB-20	Desplegar prototipo MVP en AWS (EC2/Lambda)	Deployment	6
PB-21	Integrar modelo con dashboard desplegado	Deployment	6

→ Las filas resaltadas en azul corresponden al **Sprint 4** (esta iteración): PB-13, PB-14 y PB-15.

Sprint 4: Sprint Planning

Alcance: Modeling avanzado — optimizar, ensamblar y validar

Meta del Sprint

Optimizar los **hiperparámetros** de los 2-3 candidatos del Sprint 3, explorar **ensambles y stacking**, y producir un **modelo final validado** en el test set, listo para la evaluación de Business Value en el Sprint 5.

Duración

- 1 semana de trabajo colaborativo
- Daily stand-ups (async vía GitHub Issues)
- Sprint Review al final de la semana

Sprint Backlog (3 ítems grandes)

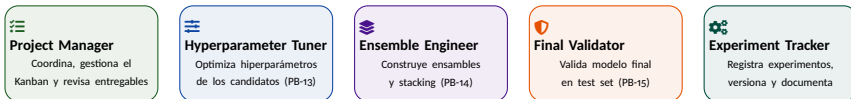
- PB-13** Optimizar hiperparámetros de los candidatos (GridSearch, RandomizedSearch, Optuna).
- PB-14** Aplicar técnicas avanzadas: ensambles (Voting, Bagging), stacking y gradient boosting (XGBoost, LightGBM).
- PB-15** Validar el modelo final en el test set y confirmar generalización.

Gestión

- Mismo repositorio GitHub de Sprints 1-3
- Cada sub-tarea = 1 Issue en el board Kanban
- Ramas por feature: `feat/PB-XX`

Roles del Equipo: 5 Personas

Roles adaptados al Sprint 4 (Modeling Avanzado)



Nota importante

Los roles **rotan** nuevamente respecto al Sprint 3: cada miembro del equipo asume un rol distinto para ganar experiencia en la fase de Modeling avanzado. Todos colaboran y revisan el trabajo de los demás vía **Pull Requests**.

Rol 1: Project Manager

 Coordinación, planificación y seguimiento

Tareas del Sprint 4

1. Crear los Issues del Sprint 4 (PB-13, PB-14, PB-15) y sub-issues por tarea.
2. Reorganizar el board Kanban (cerrar Sprint 3, abrir Sprint 4).
3. Asignar responsables y revisores cruzados a cada Issue.
4. Verificar que los baselines del Sprint 3 están disponibles y reproducibles.
5. Coordinar la secuencia: tuning → ensambles → validación final.
6. Preparar el Sprint Review: demo del modelo final y mejora vs. baseline.

Estructura del Repo (Sprint 4)

```
proyecto/  
+-- data/  
|   +-- raw/  
|   +-- interim/  
|   +-- processed/  
+-- notebooks/  
|   +-- 12_hyperparam_tuning.ipynb  
|   +-- 13_ensembles.ipynb  
|   +-- 14_final_validation.ipynb  
+-- src/  
|   +-- preprocessing.py  
|   +-- models.py  
|   +-- tuning.py  
+-- models/  
|   +-- baseline_*.pkl  
|   +-- tuned_*.pkl  
|   +-- final_model.pkl  
+-- environment.yml
```

Rol 2: Hyperparameter Tuner

Optimización de hiperparámetros (PB-13)

Tareas del Sprint 4

1. Definir los **espacios de búsqueda** de hiperparámetros para cada candidato del Sprint 3.
2. Aplicar **GridSearchCV** o **RandomizedSearchCV** con cross-validation stratified.
3. Explorar **Optuna** para búsqueda bayesiana más eficiente.
4. Comparar el rendimiento **antes vs. después** del tuning para cada candidato.
5. Persistir los modelos tuneados con joblib en `models/tuned_*.pkl`.
6. Documentar los mejores hiperparámetros encontrados y el proceso de búsqueda.

Ejemplo: GridSearchCV

```
from sklearn.model_selection import (
    GridSearchCV, StratifiedKFold)
from sklearn.ensemble import (
    RandomForestClassifier)
import joblib

cv = StratifiedKFold(
    n_splits=5, shuffle=True,
    random_state=42)

param_grid = {
    'clf__n_estimators': [100, 200, 500],
    'clf__max_depth': [5, 10, 20, None],
    'clf__min_samples_split': [2, 5, 10],
    'clf__min_samples_leaf': [1, 2, 4],
}

grid = GridSearchCV(
    pipe_rf, param_grid,
    cv=cv, scoring='f1',
    n_jobs=-1, verbose=1)

grid.fit(X_train, y_train)
print(f"Mejor F1: {grid.best_score_:.3f}")
print(f"Params: {grid.best_params_}")

joblib.dump(grid.best_estimator_,
    'models/tuned_rf.pkl')
```

Rol 2: Hyperparameter Tuner — Optuna

 Búsqueda bayesiana con Optuna

¿Por qué Optuna?

- **GridSearch** prueba todas las combinaciones: exhaustivo pero lento ($O(n^k)$).
- **RandomizedSearch** muestrea al azar: más rápido pero sin dirección.
- **Optuna** usa optimización bayesiana (TPE): aprende de resultados previos para explorar el espacio de forma inteligente.
- Soporta **pruning**: descarta trials que no prometen.
- Genera visualizaciones automáticas del proceso de búsqueda.

🔗 Ejemplo: Optuna

```
import optuna
from sklearn.model_selection import (
    cross_val_score)

def objective(trial):
    params = {
        'n_estimators': trial.suggest_int(
            'n_estimators', 50, 500),
        'max_depth': trial.suggest_int(
            'max_depth', 3, 30),
        'min_samples_split': trial.suggest_int(
            'min_samples_split', 2, 20),
        'learning_rate': trial.suggest_float(
            'learning_rate', 0.01, 0.3,
            log=True),
    }
    from sklearn.ensemble import (
        GradientBoostingClassifier)
    clf = GradientBoostingClassifier(
        **params, random_state=42)
    score = cross_val_score(
        clf, X_train, y_train,
        cv=cv, scoring='f1').mean()
    return score

study = optuna.create_study(
    direction='maximize')
study.optimize(objective, n_trials=50)
print(study.best_params)
```

Rol 3: Ensemble Engineer

 Técnicas avanzadas de ensambles (PB-14)

Tareas del Sprint 4

1. Implementar **VotingClassifier** (hard y soft voting) combinando los mejores candidatos.
2. Implementar **BaggingClassifier** sobre los modelos que mostraron overfitting.
3. Entrenar modelos de **Gradient Boosting** avanzado:
 - XGBoost (`xgboost.XGBClassifier`)
 - LightGBM (`lightgbm.LGBMClassifier`)
4. Construir un **StackingClassifier** con los mejores modelos como base y un meta-learner.
5. Evaluar todos los ensambles con cross-validation y comparar con los baselines tuneados.
6. Documentar qué combinaciones funcionan y por qué.

Ejemplo: VotingClassifier

```
from sklearn.ensemble import (
    VotingClassifier,
    StackingClassifier)

# Soft Voting: promedio de probabilidades
voting = VotingClassifier(
    estimators=[
        ('rf', best_rf),
        ('svm', best_svm),
        ('lr', best_lr),
    ],
    voting='soft')

scores = cross_validate(
    voting, X_train, y_train,
    cv=cv, scoring=scoring)

# Stacking: meta-learner
stacking = StackingClassifier(
    estimators=[
        ('rf', best_rf),
        ('svm', best_svm),
    ],
    final_estimator=LogisticRegression(),
    cv=cv)

stacking.fit(X_train, y_train)
```

Rol 3: Ensemble Engineer — Gradient Boosting

 XGBoost y LightGBM

XGBoost

```
from xgboost import XGBClassifier

xgb = XGBClassifier(
    n_estimators=200,
    max_depth=6,
    learning_rate=0.1,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42,
    eval_metric='logloss',
    use_label_encoder=False)

scores_xgb = cross_validate(
    xgb, X_train, y_train,
    cv=cv, scoring=scoring)

print(f"XGBoost F1: "
      f"{scores_xgb['test_f1'].mean():.3f}")
```

LightGBM

```
from lightgbm import LGBMClassifier

lgbm = LGBMClassifier(
    n_estimators=200,
    max_depth=6,
    learning_rate=0.1,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42,
    verbose=-1)

scores_lgbm = cross_validate(
    lgbm, X_train, y_train,
    cv=cv, scoring=scoring)

print(f"LightGBM F1: "
      f"{scores_lgbm['test_f1'].mean():.3f}")
```

Ventajas de Gradient Boosting

XGBoost y LightGBM suelen ser los algoritmos más competitivos en datos tabulares. LightGBM es más rápido en datasets grandes; XGBoost ofrece más control de regularización. Ambos soportan **early stopping** para evitar overfitting.

Rol 4: Final Validator

 Validación final del modelo seleccionado (PB-15)

Tareas del Sprint 4

1. Seleccionar el **mejor modelo** del Sprint 4 (tuneado o ensamble) basado en la métrica principal.
2. Evaluar el modelo final en el **test set** (usado por primera y única vez).
3. Generar el **classification report** y **confusion matrix** definitivos.
4. Calcular **intervalos de confianza** de las métricas usando bootstrap.
5. Comparar el rendimiento final con el **baseline** del Sprint 3 para cuantificar la mejora.
6. Documentar si el modelo cumple los **criterios de éxito** definidos en el Sprint 1.

Validación en Test Set

```
from sklearn.metrics import (
    classification_report,
    roc_auc_score)
import numpy as np

# Evaluacion FINAL en test set
# (una sola vez!)
y_pred = best_model.predict(X_test)
y_proba = best_model.predict_proba(
    X_test)[:, 1]

print(classification_report(
    y_test, y_pred))
print(f"AUC-ROC: "
      f"{roc_auc_score(y_test, y_proba):.3f}"
      ")")

# Bootstrap para intervalos de confianza
def bootstrap_metric(y_true, y_pred,
                    metric_fn, n=1000):
    scores = []
    for _ in range(n):
        idx = np.random.choice(
            len(y_true), len(y_true))
        scores.append(
            metric_fn(y_true[idx],
                      y_pred[idx]))
    return np.percentile(scores, [2.5,
                                97.5])

ci = bootstrap_metric(
    y_test, y_proba, roc_auc_score)
print(f"AUC 95% CI: [{ci[0]:.3f}, {ci[1]:.3f}]"
      ")")
```

Rol 4: Final Validator — Mejora vs. Baseline

 Cuantificar el impacto de la optimización

Tabla de Mejora

Generar una tabla que compare directamente las métricas del **baseline** (Sprint 3) con las del **modelo optimizado**

	Métrica	Baseline	Tuned	Δ
(Sprint 4):	F1	0.83	0.89	+7.2 %
	AUC-ROC	0.90	0.94	+4.4 %
	Recall	0.81	0.87	+7.4 %
	Precision	0.85	0.91	+7.1 %

Esta tabla es clave para el **Sprint 5**: el Business Analyst la usará para evaluar si la mejora justifica el costo del modelo más complejo.

Ejemplo

```
import pandas as pd

# Comparar baseline vs tuned
comparison = pd.DataFrame({
    'Metrica': ['F1', 'AUC-ROC',
               'Recall', 'Precision'],
    'Baseline (Sprint 3)': [
        0.83, 0.90, 0.81, 0.85],
    'Tuned (Sprint 4)': [
        0.89, 0.94, 0.87, 0.91],
})

comparison['Mejora (%)'] = (
    (comparison['Tuned (Sprint 4)']
     - comparison['Baseline (Sprint 3)'])
    / comparison['Baseline (Sprint 3)']
    * 100
).round(1)

comparison.style.highlight_max(
    subset=['Tuned (Sprint 4)'],
    color='lightgreen')
```

Rol 5: Experiment Tracker

 Registro de experimentos y versionado de modelos

Tareas del Sprint 4

1. Extender `src/models.py` con funciones de tuning y crear `src/tuning.py` con utilidades de búsqueda de hiperparámetros.
2. Registrar **cada experimento** de tuning: modelo, hiperparámetros probados, métricas obtenidas, tiempo de ejecución.
3. Persistir todos los modelos tuneados y el **modelo final** con `joblib` y versionar con **DVC**.
4. Validar que el modelo final se puede cargar y predecir correctamente.
5. Actualizar el `experiments_log.csv` con los resultados del Sprint 4.
6. Documentar el flujo completo: desde baselines hasta modelo final optimizado.

`src/tuning.py`

```
import joblib
import time
import pandas as pd
from sklearn.model_selection import (
    GridSearchCV)

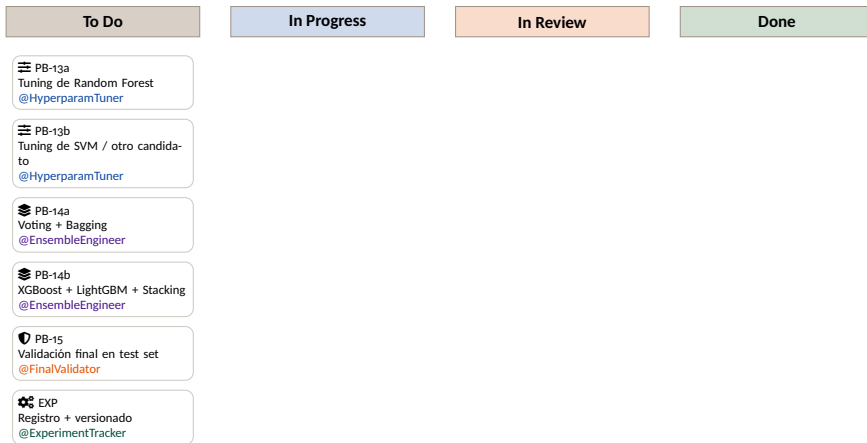
def tune_model(pipe, param_grid, X, y,
               cv, scoring='f1', name=''):
    """Tunea hiperparámetros y registra."""
    start = time.time()
    grid = GridSearchCV(
        pipe, param_grid,
        cv=cv, scoring=scoring,
        n_jobs=-1, verbose=0)
    grid.fit(X, y)
    elapsed = time.time() - start

    # Persistir mejor modelo
    path = f'models/tuned_{name}.pkl'
    joblib.dump(grid.best_estimator_, path)

    return {
        'model': name,
        'best_params': grid.best_params_,
        'best_score': grid.best_score_,
        'time_seconds': round(elapsed, 1),
        'path': path,
    }
```


Tablero Kanban en GitHub Projects

Visualización del flujo de trabajo del Sprint 4



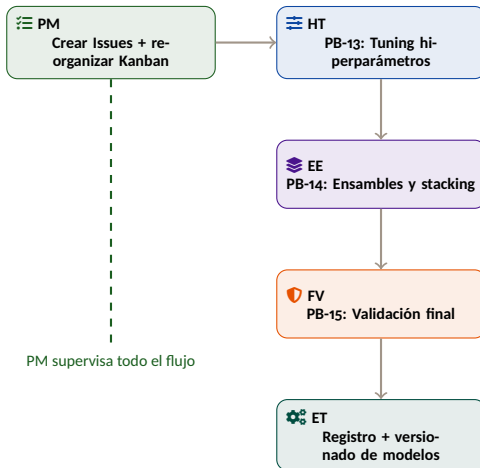
Flujo de Trabajo en el Kanban

Flujo

Cada Issue se mueve: **To Do** → se crea rama `feat/PB-XX` → **In Progress** → se abre Pull Request → **In Review** → se aprueba y mergea → **Done**. El Experiment Tracker integra al final, una vez los modelos están tuneados y validados.

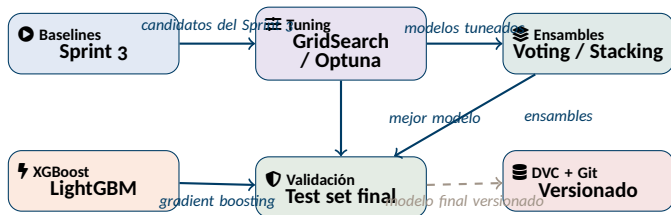
Dependencias entre Tareas

Orden sugerido de ejecución en el Sprint 4



Herramientas del Sprint 4: ¿Cómo encajan juntas?

Visión integrada para Modeling Avanzado



💡 Idea clave

Los baselines del Sprint 3 son el punto de partida. El Sprint 4 los mejora vía tuning, los combina en ensambles, y valida el mejor modelo en el test set. El resultado es un **modelo final listo** para la evaluación de Business Value.

Tip 1: GridSearch vs. RandomizedSearch vs. Optuna

 Elegir la estrategia de búsqueda correcta

¿Cuándo usar cuál?

- **GridSearchCV**: espacio pequeño (<100 combinaciones). Exhaustivo, ideal para pocos hiperparámetros.
- **RandomizedSearchCV**: espacio grande. Muestrea n combinaciones al azar. Más rápido, sorprendentemente efectivo.
- **Optuna**: espacio muy grande o cuando se necesita búsqueda inteligente. Usa TPE (Tree-structured Parzen Estimator) para dirigir la búsqueda.

Regla práctica

Empezar con RandomizedSearchCV (rápido) para acotar la región prometedora, luego refinar con GridSearchCV u Optuna en esa región.

RandomizedSearchCV

```
from sklearn.model_selection import (
    RandomizedSearchCV)
from scipy.stats import randint, uniform

param_dist = {
    'clf__n_estimators': randint(50, 500),
    'clf__max_depth': randint(3, 30),
    'clf__min_samples_split':
        randint(2, 20),
    'clf__min_samples_leaf':
        randint(1, 10),
}

random_search = RandomizedSearchCV(
    pipe_rf, param_dist,
    n_iter=100,
    cv=cv, scoring='f1',
    n_jobs=-1, random_state=42)

random_search.fit(X_train, y_train)
print(f"Mejor F1: "
      f"{random_search.best_score_:.3f}")
```

Tip 2: Ensamblés — combinar es mejor que elegir

 Cómo y por qué combinar modelos

? Tipos de ensambles

- **Voting:** combina predicciones por voto (hard) o promedio de probabilidades (soft). Funciona bien cuando los modelos base son **diversos**.
- **Bagging:** entrena el mismo modelo en subsets aleatorios de datos. Reduce **varianza** (overfitting).
- **Boosting:** entrena modelos secuencialmente, cada uno corrigiendo los errores del anterior. Reduce **sesgo**.
- **Stacking:** usa las predicciones de modelos base como features para un meta-learner. Más flexible, más riesgo de overfitting.

💡 ¿Cuándo sirve cada uno?

Técnica	Usar cuando...
Voting	Modelos base son buenos y diversos
Bagging	Un modelo tiene alta varianza (overfitting)
Boosting	Se necesita más precisión en errores difíciles
Stacking	Se quiere combinar modelos de tipos muy distintos

⚠ Cuidado

Más complejidad no siempre es mejor. Si un modelo tuneado ya cumple los criterios de éxito, preferir la solución más simple (Occam's Razor).

Tip 3: Early Stopping en Gradient Boosting

 Detener el entrenamiento antes de overfittear

? ¿Qué es Early Stopping?

- En gradient boosting, cada árbol nuevo reduce el error de entrenamiento.
- Pero después de cierto punto, el error de **validación** empieza a subir → **overfitting**.
- **Early stopping** monitorea el error de validación y detiene el entrenamiento cuando deja de mejorar.
- Ahorra tiempo y mejora la generalización.

</> Ejemplo

```
from xgboost import XGBClassifier
from sklearn.model_selection import (
    train_test_split)

# Split interno para early stopping
X_tr, X_val, y_tr, y_val = (
    train_test_split(
        X_train, y_train,
        test_size=0.2,
        stratify=y_train,
        random_state=42))

xgb = XGBClassifier(
    n_estimators=1000,
    learning_rate=0.05,
    early_stopping_rounds=50,
    eval_metric='logloss',
    random_state=42)

xgb.fit(X_tr, y_tr,
        eval_set=[(X_val, y_val)],
        verbose=False)

print(f"Árboles usados: "
      f"{xgb.best_iteration}")
```

Tip 4: Feature Importance post-tuning

⚡ Entender qué features importan en el modelo final

❓ ¿Por qué analizar importancia?

- Ayuda a **interpretar** el modelo: ¿qué variables impulsan las predicciones?
- Permite detectar si el modelo depende de features **ruidosas o irrelevantes**.
- Es insumo clave para el **Sprint 5**: el Business Analyst necesita explicar al negocio *por qué* el modelo decide lo que decide.
- Puede revelar oportunidades de **simplificar** el modelo (menos features = más rápido y robusto).

⚡ Ejemplo

```
import matplotlib.pyplot as plt
import pandas as pd

# Feature importance del modelo final
importances = (
    best_model.feature_importances_
)
feature_names = X_train.columns

fi = pd.DataFrame({
    'feature': feature_names,
    'importance': importances
}).sort_values(
    'importance', ascending=False)

fig, ax = plt.subplots(figsize=(8, 5))
ax.barh(fi['feature'][:15],
        fi['importance'][:15])
ax.set_xlabel('Importancia')
ax.set_title('Top 15 Features')
ax.invert_yaxis()
plt.tight_layout()
plt.show()
```


Errores comunes en Modeling Avanzado

 Qué evitar y cómo resolverlo

	Error común	Solución
⊘	Tunear sin cross-validation (evaluar solo en un split)	Siempre usar GridSearchCV o equivalente con CV estratificado
⊘	Usar el test set para elegir hiperparámetros	El test set solo se usa una vez al final ; usar CV para tuning
⊘	Espacios de búsqueda demasiado grandes con GridSearch	Usar RandomizedSearchCV u Optuna para espacios grandes
⊘	Agregar complejidad sin medir la mejora vs. baseline	Siempre comparar cada mejora contra el baseline del Sprint 3
⊘	No usar <code>random_state</code> en los modelos y CV	Fijar semilla para reproducibilidad en todo el pipeline
⊘	Elegir el modelo más complejo sin considerar interpretabilidad	Preferir modelos más simples si el rendimiento es comparable

Entregables del Sprint 4

Lo que cada rol debe entregar al final de la semana

Rol	Entregable	Formato
PM	Issues del Sprint 4 cerrados, board Kanban actualizado, Sprint Review preparado	GitHub Project
HT	Notebook con tuning de candidatos, tablas de mejora vs. baseline, modelos tuneados persistidos	<code>12_hyperparam_tuning.ipynb</code>
EE	Notebook con ensambles (Voting, Stacking, XGBoost, LightGBM), comparación con modelos individuales	<code>13_ensembles.ipynb</code>
FV	Notebook con validación final en test set, classification report definitivo, intervalos de confianza	<code>14_final_validation.ipynb</code>
ET	Módulo <code>src/tuning.py</code> , registro de experimentos actualizado, modelo final versionado con DVC	<code>src/tuning.py + experiments_log.csv</code>

Definition of Done (DoD)

Criterios que cada tarea debe cumplir para considerarse terminada

✓ Criterios Generales

- ☐ Código en rama `feat/PB-XX` con Pull Request aprobado y mergeado a `main`.
- ☐ Notebook ejecutable de principio a fin sin errores (*Restart & Run All*).
- ☐ Celdas markdown documentando **cada decisión** de modelado.
- ☐ El entorno se reproduce con `conda env create -f environment.yml`.
- ☐ Issue correspondiente movido a "Done" en el board.

★ Criterios específicos del Sprint 4

- ☐ Hiperparámetros optimizados para al menos 2 candidatos del Sprint 3.
- ☐ Al menos 2 técnicas de ensamble implementadas y evaluadas.
- ☐ Tabla comparativa: baseline vs. tuned vs. ensamble con mejora porcentual.
- ☐ Modelo final evaluado en test set (una sola vez) con intervalos de confianza.
- ☐ Modelo final persistido como `models/final_model.pkl` y versionado con DVC.
- ☐ Registro de experimentos actualizado con todos los resultados del Sprint 4.

Visión General: 6 Sprints hasta el Despliegue

Del entendimiento del negocio al MVP desplegado en AWS

Sprint	Fase CRISP-DM	Objetivo principal
1	Business + Data Underst.	Comprender el problema y explorar los datos
2	Data Preparation	Limpiar, transformar y crear features
3	Modeling	Entrenar modelos baseline y comparar
4	Modeling	Optimizar hiperparámetros y modelos avanzados
5	Evaluation	Evaluar Business Value y documentar resultados
6	Deployment	Desplegar el mejor modelo como MVP en AWS

Meta del Sprint 4

Entregar un **modelo final optimizado y validado** que mejore significativamente los baselines del Sprint 3, con evidencia cuantitativa de la mejora y listo para que el Sprint 5 evalúe su **impacto de negocio**.

Resumen del Sprint 4

¿Qué?

Modeling avanzado: tuning, ensambles y validación final

¿Quién?

5 roles: PM, HT, EE, FV, ET

¿Cómo?

GridSearch + Optuna
+ XGBoost + Stacking



Entregable

3 Notebooks + `src/tuning.py`
+ modelo final versionado + log actualizado



Siguiente

Sprint 5: Evaluación + Business Value

! Recordar

- El Sprint 4 **optimiza** los candidatos del Sprint 3. Siempre medir la mejora vs. baseline.
- El **test set** solo se usa una vez al final para la validación definitiva del modelo final.
- Más complejidad no siempre es mejor: preferir el modelo más simple que cumpla los **criterios de éxito**.
- Documentar cada experimento: hiperparámetros, métricas, tiempo y justificación.

Checklist de Inicio

Pasos inmediatos para comenzar el Sprint 4

1. **PM:** Cerrar el Sprint 3 en el board y crear los Issues del Sprint 4 (PB-13, PB-14, PB-15 y sub-issues).
2. **PM:** Reasignar roles para que cada miembro experimente la fase de Modeling avanzado.
3. **Todos:** `git pull && dvc pull` para sincronizar el estado del Sprint 3.
4. **Todos:** Verificar que los baselines del Sprint 3 se cargan correctamente:
`joblib.load('models/baseline_rf.pkl')`.
5. **HT:** Iniciar `12_hyperparam_tuning.ipynb` en rama `feat/PB-13`, definir espacios de búsqueda y tunear candidatos.
6. **EE:** Una vez tuneados los modelos, implementar ensambles y gradient boosting en `13_ensembles.ipynb` (rama `feat/PB-14`).
7. **FV:** Tras los ensambles, validar el mejor modelo en test set en `14_final_validation.ipynb` (rama `feat/PB-15`).

Checklist de Inicio (cont.)

Pasos inmediatos para comenzar el Sprint 4

8. **ET:** Crear `src/tuning.py`, versionar modelos tuneados y el modelo final con DVC, y actualizar el registro de experimentos (rama `feat/EXP`).
9. **Todos:** Sprint Review — presentar el modelo final, la mejora vs. baseline y los candidatos listos para el Sprint 5.



Diplomado de Especialización en **Data Analytics**